



Laços, Arrays e Métodos

Aula 04 · Bloco 1 — Fundamentos

A última aula antes da POO — e o problema que ela vem resolver

Nesta aula

1. `while` e `do-while`
2. `for`, `break`, `continue`
3. **Arrays** — muitos valores, um nome
4. **Métodos** — dando nome a um pedaço de lógica
5. **O limite** do que fizemos até aqui

while e do-while

```
while (contador <= 5) {           // testa ANTES de cada repetição
    contador++;                  // 🤪 esquecer = laço infinito
}

do {                             // executa PELO MENOS UMA VEZ
    opcao = scanner.nextInt();
} while (opcao != 0);
```

⚠️ Laço infinito = cursor piscando para sempre. **Ctrl + C** interrompe. Depois procure o que deveria mudar a condição.

for — quando você sabe quantas vezes

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("Repetição " + i);  
}  
//   ↑ início   ↑ condição ↑ passo
```

Tudo o que o **while** faz em quatro linhas, o **for** faz em uma.

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 != 0) continue; // ímpar? pula esta volta  
    if (i > 8) break;         // passou de 8? encerra o laço  
    System.out.println(i);    // 2 4 6 8  
}
```

Arrays: muitos valores, um nome

```
double[] notas = new double[4];    // 4 posições, todas 0.0
notas[0] = 8.5;                    // índices começam em ZERO
String[] nomes = {"Ana", "Bruno", "Carla"};

nomes.length    // 3 – PROPRIEDADE, sem parênteses!
notas[4]         // ✨ ArrayIndexOutOfBoundsException
```

⚠️ Array de tamanho 4 tem índices **0, 1, 2, 3**. O último é sempre `length - 1` — por isso a condição do `for` usa `<`, nunca `<=`.

Percorrendo

```
// for clássico – quando você precisa do ÍNDICE
for (int i = 0; i < notas.length; i++) {
    System.out.printf("Nota %d: %.1f%n", i + 1, notas[i]);
}

// for-each – quando só quer os VALORES (mais limpo)
for (double nota : notas) {           // "para cada nota em notas"
    soma += nota;
}
```

💡 Array tem **tamanho fixo**. Cadastro que cresce pede `ArrayList` — aula 09.

Métodos

Um `main` de 200 linhas é impossível de entender e de testar.

Método é um bloco de código **com nome**, que recebe dados e talvez devolva resultado.

A anatomia de um método

```
public static double calcularMedia(double[] valores) { ... }
```

Parte

Quer dizer

`public`

visível de fora

`static`

pertence à classe, não a um objeto

`double`

o tipo do que devolve (`void` = nada)

`(double[] valores)`

os parâmetros

Por que vale a pena

 **Nome é documentação.** `calcularMedia(notas)` se explica sozinho;

 **Reuso.** Escreveu uma vez, chama quantas quiser;

 **Conserto local.** Bug na média? Existe **um** lugar para olhar.

 Método com retorno precisa de `return` em **todos** os caminhos. Senão:
`error: missing return statement`.

Sobrecarga: mesmo nome, assinaturas diferentes

```
public static double somar(double a, double b) { ... }  
public static double somar(double a, double b, double c) { ... }  
  
somar(2, 3);           // usa a primeira  
somar(2, 3, 4);        // usa a segunda
```

O compilador escolhe pelo **número e tipo** dos argumentos.

💡 É por isso que `System.out.println` aceita texto, número ou boolean: são várias versões sobrecarregadas.

Escopo: onde cada variável existe

```
int fora = 10;

for (int i = 0; i < 3; i++) {
    int dentro = i * 2;
    System.out.println(fora + dentro);    // ✅ enxerga as duas
}

System.out.println(dentro);    // ❌ morreu ao fechar o for
```

Cada método tem seu próprio espaço. Métodos conversam por **parâmetro e retorno** — nunca por variável compartilhada.

E agora, o limite

Cadastrar 3 alunos com nome, matrícula e notas:

```
String[] nomes = new String[3];  
String[] matriculas = new String[3];  
double[][] notas = new double[3][3];
```

Três arrays paralelos que você precisa
sincronizar na mão.

O que dá errado

Ordenou os nomes? As notas ficaram trocadas.

Removeu um aluno? Tem que lembrar de remover em **três lugares**.

E a média é um método solto, longe dos dados dele.

Falta uma forma de dizer:

*"um aluno **é** um nome, uma matrícula e três notas — e **sabe** calcular a própria média."*

É exatamente isso que a próxima aula traz. 🙌

Exercícios da aula

Na pasta `aula-04/` :

1. `Tabuada.java` — com `printf`, e depois `for` dentro de `for`;
2. `Estatisticas.java` — maior, menor, soma e média, cada um em seu método;
3. `Menu.java` — `do-while` com histórico num array, sem quebrar no limite;
4. `Refatorar.java` — extraia 3 métodos; o `main` final com no máximo 6 linhas;
5. **Desafio** 🌶️ `Boletim.java` — arrays paralelos. E anote **o que foi mais chato**.

Próxima aula

Aula 05 — Classes e Objetos

O começo da POO —
e a solução para os arrays paralelos.